# Homework — 2.2.1 Programming Techniques — ANSWER SHEET

*Separate answer sheet / mark scheme — keep for marking.*

## Mark scheme

**Part A (14 marks)**

**1. [2]** — 1 mark: **count-controlled** repeats a **fixed, known** number of times (number of repeats known in advance) — construct `for...next`. 1 mark: **condition-controlled** repeats **until a condition changes** (number of repeats not known in advance) — construct `while` or `do...until`.

**2. [2]** — 1 mark: `do...until` is **post-conditioned** (tests at the **end**), so the body always runs **at least once**. 1 mark: input validation needs the prompt/read to execute at least once before a value can be tested, so the at-least-once behaviour fits validation (a `while` could run zero times if no value yet exists).

**3. [3]** — Trace of `power(2, 4)` (frames pushed, then popped LIFO):

```
power(2,4) = 2 * power(2,3)
power(2,3) = 2 * power(2,2)
power(2,2) = 2 * power(2,1)
power(2,1) = 2 * power(2,0)
power(2,0) = 1            (base case, exp = 0)
```

Unwinding: 2*1* = *2* → 2*2* = 4 → 2*4* = *8* → 2*8* = **16**. - 1 mark: base case returns 1 at `exp = 0`. - 1 mark: correct returns shown at each level (2, 4, 8, 16) / correct multiplication on unwind. - 1 mark: final answer **16**.

**4.** - (a) **[2]** — Output line 1: **0** (1). Output line 2: **8** (1). - (b) **[2]** — 1 mark: inside `reset()` the assignment creates/uses a **local** variable `count` (scope = the procedure only) holding 0, so line 1 prints 0. 1 mark: the **global** `count` is unchanged because the local **shadows** it but does not alter it, so after the call line 2 prints 8.

**5. [2]** — 1 mark: after `scaleByVal(total)`, **total = 10** because a **copy** of the value is passed, so the original is **unchanged**. 1 mark: after `scaleByRef(total)`, **total = 20** because the **memory address** of `total` is passed, so updating `n` updates the original. *(Mark the consequence — unchanged / changed — not merely "a copy is made".)*

**6. [1]** — **Encapsulation** = bundling data (attributes) and methods together in a class and keeping attributes **private**, accessed only via public methods (data hiding). *(Award the mark for a correct definition; benefits such as protecting data integrity / preventing invalid external changes / easier maintenance reinforce but are not required.)*

**Part B (6 marks)**

**7. [2]** — 1 mark: **abstraction** = removing/hiding unnecessary detail to focus on what matters. 1 mark (any one): makes the problem simpler/more manageable; reduces complexity; allows a general model/solution; lets the solver ignore irrelevant detail.

**8. [2]** — 1 mark: stack = **LIFO** (Last In, First Out). 1 mark: queue = **FIFO** (First In, First Out).

**9. [2]** — 1 mark: **decomposition** = breaking a complex problem into smaller, more manageable sub-problems. 1 mark: so each sub-problem can be solved/tested separately (and the work divided / reused). *(Accept a clear single developed point as 1 + 1.)*

**Total: 14 + 6 = 20**